

ĐỀ BÀI THỰC HÀNH: XÂY DỰNG MẠNG NƠ-RON NHÂN TẠO TỪ SCRATCH (USING NUMPY) TRÊN TẬP MNIST

Mục tiêu:

- Hiểu rõ các thành phần cơ bản của mạng nơ-ron (tầng fully-connected, hàm kích hoạt, hàm mất mát, lan truyền tiến (forward), lan truyền ngược (backward), cập nhật tham số).
- Lập trình một mạng MLP (multi-layer perceptron) đơn giản từ đầu bằng NumPy.
- Huấn luyện mạng trên bộ dữ liệu MNIST (images 28×28 , 10 lớp) và đánh giá.

Yêu cầu:

1. Cài đặt các hàm: softmax, cross-entropy loss, ReLU (hoặc Sigmoid/tanh), cùng gradient tương ứng.
2. Cài đặt lớp Dense (fully-connected) với forward/backward.
3. Cài đặt optimizer SGD (có momentum optional).
4. Hoàn thiện SimpleMLP (forward/backward).
5. Huấn luyện ít nhất một mạng 2-lớp ẩn (ví dụ: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$). Báo cáo độ chính xác (accuracy) trên tập validation/test.

Khung code (sinh viên hoàn thiện những chỗ TODO): """

```
import numpy as np
import os
import time
```

Utility: load MNIST

```
def load_mnist_npz(path: str = 'data/mnist.npz'):
    if not os.path.exists(path):
        raise FileNotFoundError("Chưa có file mnist.npz, hay tải
va dat vao thu muc data/")
    f = np.load(path)
    x_train, y_train = f['x_train'], f['y_train']
    x_test, y_test = f['x_test'], f['y_test']
    x_train = x_train.astype(np.float32) / 255.0
```

```
x_test = x_test.astype(np.float32) / 255.0
x_train = x_train.reshape(x_train.shape[0], -1)
x_test = x_test.reshape(x_test.shape[0], -1)
return x_train, y_train, x_test, y_test
```

Activation functions (SV hoàn thiện TODO)

```
def softmax(z):
    # TODO: viet softmax on dinh so hoc
    pass

def relu(z):
    # TODO: viet ham ReLU
    pass

def relu_grad(z):
    # TODO: dao ham cua ReLU
    pass

def sigmoid(z):
    # TODO: viet ham sigmoid
    pass

def sigmoid_grad(a):
    # TODO: dao ham cua sigmoid (a = sigmoid(z))
    pass
```

Loss function

```
def cross_entropy_loss(probs, y_true):
    # TODO: viet ham cross-entropy
    pass
```

Dense layer class

```
class Dense:
    def __init__(self, in_dim, out_dim, weight_scale=1e-2):
        self.W = np.random.randn(in_dim, out_dim) * weight_scale
        self.b = np.zeros((1, out_dim))
```

```

        self.x = None
        self.dW = None
        self.db = None

    def forward(self, x):
        # TODO: implement forward
        pass

    def backward(self, dout):
        # TODO: implement backward
        pass

```

Simple MLP model

```

class SimpleMLP:
    def __init__(self, input_dim, hidden_dims, num_classes,
activation='relu'):
        # TODO: khoi tao cac layer
        pass

    def forward(self, x):
        # TODO: lan truyen tien
        pass

    def compute_loss_and_probs(self, x, y):
        # TODO: tra ve loss va xac suat du doan
        pass

    def backward(self, probs, y):
        # TODO: lan truyen nguoc
        pass

    def get_params_and_grads(self):
        # TODO: tra ve list params va list grads
        pass

```

Optimizer

```

class SGD:
    def __init__(self, params, lr=1e-2):
        self.params = params
        self.lr = lr

```

```
def step(self, grads):
    # TODO: cap what tham so theo SGD
    pass
```

Training loop

```
def accuracy(probs, y):
    preds = np.argmax(probs, axis=1)
    return np.mean(preds == y)

def train(model, x_train, y_train, x_val, y_val, epochs=10,
          batch_size=128, lr=1e-2):
    params, _ = model.get_params_and_grads()
    optimizer = SGD(params, lr=lr)

    num_samples = x_train.shape[0]
    steps_per_epoch = int(np.ceil(num_samples / batch_size))

    for epoch in range(1, epochs+1):
        perm = np.random.permutation(num_samples)
        x_shuffled = x_train[perm]
        y_shuffled = y_train[perm]

        epoch_loss = 0.0
        epoch_acc = 0.0

        t0 = time.time()
        for step in range(steps_per_epoch):
            start = step * batch_size
            end = min((step+1) * batch_size, num_samples)
            xb = x_shuffled[start:end]
            yb = y_shuffled[start:end]

            loss, probs = model.compute_loss_and_probs(xb, yb)
            epoch_loss += loss * xb.shape[0]
            epoch_acc += accuracy(probs, yb) * xb.shape[0]

            # backward + update
            model.backward(probs, yb)
            _, grads = model.get_params_and_grads()
            optimizer.step(grads)

        epoch_loss /= num_samples
        epoch_acc /= num_samples

    val_loss, val_probs = model.compute_loss_and_probs(x_val,
                                                         y_val)
```

```
val_acc = accuracy(val_probs, y_val)

t1 = time.time()
print(f"Epoch {epoch}/{epochs} - loss: {epoch_loss:.4f} -
acc: {epoch_acc:.4f} - val_loss: {val_loss:.4f} - val_acc: {
val_acc:.4f} - time: {t1-t0:.2f}s")
```

Main

```
if __name__ == '__main__':
    try:
        x_train, y_train, x_test, y_test = load_mnist_npz('data/
mnist.npz')
    except FileNotFoundError as e:
        print(e)
        raise

    # TODO: chia train/val, khoi tao model, goi train(), danh gia
    tren test
    pass
```

TODO (SV hoàn thiện):

1. Viết các hàm kích hoạt và đạo hàm.
2. Viết cross-entropy loss.
3. Hoàn thiện forward/backward cho Dense và SimpleMLP.
4. Hoàn thiện SGD.step().
5. Đánh giá độ chính xác trên test set sau khi huấn luyện.